

13.1 Constraint Programming

In the previous two chapters we have seen, with the two declarative paradigms there discussed, two major steps toward the “holy grail of programming: the user states the problem, the computer solves it” [1]. Constraint programming provides a further, significative step in this direction.

Indeed, as we have already seen in the previous chapters, the very idea of declarative programming is that the programmer specifies *what* has to be done, and the programming language system autonomously discovers *how*. An essential component of this declarative approach is the linguistic tool that the programmer can use to describe the *what*, that is, the problem to be solved. With functional languages this tool is the function. This is certainly a very general concept useful for describing many kind of computational problems, however it suffers the limitation of “directionality”: consider simply the very definition of a function as a rule which associates to an element of the domain at most one element of the codomain; from this definition it is clear that if we want to change the direction of application we need to use another function (which, of course, is the inverse). As an example, consider the famous definition of speed by Galileo Galilei described by the equation

$$v = \frac{d}{t} \quad (13.1)$$

where v is speed, d is distance and t is time. If, for a fixed speed v , we want to know the distance which is covered in time t we can derive from the previous equation the function

$$Distance(t) = v \times t.$$

However, if we want to derive (for the same fixed speed v) the time needed to cover the distance d we need a different function

$$Time(d) = \frac{d}{v}$$

which is the inverse of the function *Distance*. Both these two functions are contained in Eq. 13.1, however if we use a functional language we cannot have a unique construct for denoting both of them.

The same problem is shared by imperative languages, since also assignment has a directional nature: if we want to compute the distance we need the assignment

$$D = V * T$$

(where *D*, *V* and *T* are variables with the obvious meaning) while if we want to compute the time we need a different assignment command:

$$T = D / V.$$

The key observation here is that Eq. 13.1 expresses a *relation*¹ or, in other terms, a *constraint* on the quantities *v*, *d* and *t*, which relates their values by a specific mathematical law. In order to overcome the limitation shown by the previous example we would then need a language which allows us to use relations, or constraints, as “first class citizens”, meaning that relations can be written, manipulated, computed and provided as results of computation: this is exactly what a constraint programming language provides. By using such a language one could simply write the constraint

$$D == V * T$$

where *==* means equality (and not assignment), and then we could obtain the value of one variable once the values of the two others are provided. But, more than that, from such a constraint one could also obtain as the result of the computation a new constraint (i.e. a new relation) rather than a single value. For example, if we add to the previous constraint the following one

$$V == 2$$

from these two constraints we could obtain as the result of the computation the constraint

$$D == 2 * T$$

which, being a relation, provides a result describing an (infinite) set of values rather than a single value.

This ability to manipulate relations and obtain them as results of computation is essential in many applications, especially in artificial intelligence: many problems have a combinatorial nature which allow us to easily express them in terms of constraints stating different relations among different objects. Of course, once the problem is expressed as a set of constraints, we need a suitable tool which allows us to simplify them and to perform some computations in order to obtain a solution. Such a tool is called *constraint solver* (or solver for short) and its nature depends on the specific domain on which the constraints are defined. For example, if we consider conjunctions of linear equations as constraints and real numbers as the domain of

¹ For the forgetful reader we recall that an *n*-ary relation on *n* sets $D_1, D_2, \dots, D_n$ is any subset of the Cartesian product $D_1 \times D_2 \times \dots \times D_n$.

computation, then we could use the Gauss-Jordan method as the constraint solver.

The careful reader has surely noticed that, in the previous chapter, we have already seen a constraint programming paradigm where relations can be used, manipulated and computed. Indeed, if in a logic programming language we write the equations

$$X = t, Y = f(a)$$

we are considering a particular class of constraints, namely equalities over the Herbrand Universe.² More precisely, in the previous equations, the predicate symbol $=$ is interpreted as syntactic equality over the Herbrand Universe, while the comma indicates logical conjunction. These specific constraints then refer to a specific solver, the unification algorithm, which, as we have seen, is integrated in the logic programming language.

The general idea of constraint programming is to consider further domains beyond the Herbrand Universe, both numerical and symbolic ones, with the related solvers, and to integrate them with existing languages.

The development of appropriate, efficient constraint solvers is a central topic in constraint programming which, however, is beyond the scope of this book. We only notice here that they are based on some *domain specific methods* which are related to the specific domain for the values used by the constraint (for example, methods for solving linear equations, packages for linear programming, unification algorithm, etc.). These specific methods are integrated with *general methods* which aim to reduce the search space (for reaching a solution) and use both *constraint propagation* techniques based on *local consistency* and specific search methods.

Constraint propagation is a key technique for constraints on finite domains. Its idea is to use existing constraints to perform some inference in order to reduce the number of admissible values for the existing variables. In modern constraint programming systems, these techniques are thoroughly integrated with the features offered by the host programming language, thus allowing the programmer to model the problem to be solved at a very high level, while the underlying solving machinery provide powerful tools for solving the problem. In most cases, the main difficulty of the programmer in solving a concrete problem is then finding an appropriate description, or model, for it.

As previously mentioned, in the following, we will consider the integration of constraints both with logic programming and with imperative languages, since these offer the mainly used systems today.

13.1.1 Types of Problems

Constraints arise in many real life applications and have been used to model and solve a variety of problems since the '60s. The following is an incomplete list which

² Recall that this is the set of ground terms.

includes both research and commercial applications of constraint programming and should provide an idea of the practical relevance of this technology:

- analysis, simulation, verification, diagnosis of software, hardware, and industrial processes (e.g., circuit verification in major producers of electronic components);
- configuration (e.g., software configuration for mobile phones);
- design (e.g., aircraft cabin layouts);
- financial applications (e.g., investment portfolio management and optimisation for banks);
- decision support systems (e.g., systems for diagnosis in medicine);
- planning (e.g., harbour resource planner, short-term production planning in car manufacturer, parcel delivery in logistics);
- research in molecular biology, biochemistry, and bio-informatics (e.g., protein folding, genomic sequencing);
- risk management (e.g., systems for dams management);
- scheduling and timetabling (crew and aircraft scheduling in airlines, timetables of railways).

All these applications involve different domains, such as blocks, booleans, finite domains, integer numbers, intervals, real numbers, strings, terms, trees, temporal intervals and others.

Particularly relevant for many applications are finite domains which, despite the apparent simplicity—being finite the possible values, variables, and constraints, the solution can always be found in a finite time—often involve problems which are computationally very difficult—many problems are NP-complete; just consider Boolean satisfiability.

Some practical problems which can be formulated by means of constraints require us to find just one solution, no matter how “good” it is. This class of problems is usually formalised by using the notion of Constraint satisfaction problem (CSP for short) as follows.

Definition 13.1 (CSP) A Constraint Satisfaction Problem (CSP) consists of

1. a finite set $V = \{x_1, \dots, x_n\}$ of variables;
2. a set $D = \{D_1, \dots, D_n\}$ of domains where, for $i \in [1, n]$, the variable x_i assumes values in the domain D_i ;
3. a set C of constraints which specify the allowed values for variables (i.e., relations on the domains).

A *solution* is a tuple $(d_1, \dots, d_n)$ of values which satisfies all the constraints in C where, for $i \in [1, n]$, $d_i \in D_i$ is the value of x_i .

Clearly different languages and systems provide different syntax for expressing constraints, and a precise notion of satisfiability can be defined on the basis of such a syntax (in the next section we will see a notion based on first order logic). However,

we can easily define what it means for a tuple of values to satisfy a constraint by using its definition as a relation: if the constraint c is a relation on $D_1, \dots, D_n$ then $(d, \dots, d_n) \in D_1 \times \dots \times D_n$ satisfies c iff $(d_1, \dots, d_n) \in c$. So, for example, the pair of values (10,5) for the pair of variables (x, y) satisfies the constraint $x > y$ and does not satisfy the constraint $x = y$.

Example 13.1 A classic CSP problem is the Map coloring. Consider a map of central Italy describing the regions Toscana, Marche, Umbria, Lazio, Abruzzo. The task is to color each region either red, green or blue in such a way that neighboring regions have different colors. The problem can be represented by using the variables $V = \{T, M, U, L, A\}$ for representing the regions and the domain $D = \{\text{red}, \text{green}, \text{blue}\}$. If we look at the map, the constraints requiring neighboring regions having different colors can then be simply expressed as

$$C = \{T \neq M, T \neq U, T \neq L, M \neq L, M \neq U, M \neq A, U \neq L, L \neq A\}.$$

For some applications finding just one solution is not enough: among all the possible solutions to our constraints, we want to find one which maximises (or minimises) a given cost function. Consider for example the famous travelling salesperson problem: among all the different routes between two destinations, we want to find one which minimises the distance. This class of problems, which involve an objective (or cost) function which should be optimized, are called Constraint optimisation problems (COP for short) and can be defined as follows.

Definition 13.2 (COP) A Constraint Optimisation Problem (COP) consists of a CSP defined on the variables $\{x_1, \dots, x_n\}$, the constraints C and the domains $\{D_1, \dots, D_n\}$, together with an objective function $f : D_1 \times \dots \times D_n \rightarrow D$.³

A solution to the constraint optimisation problem is a solution to the CSP which optimize the function f .

We will consider constrained optimisation problem later in Sect. 13.4, when discussing constraints in the setting of imperative languages.

13.2 Constraint Logic Programs

We now discuss *Constraint logic programming*, an extension of logic programming which adds sophisticated constraint-satisfaction to the mechanisms we have seen in the previous chapter. The resulting paradigm is very interesting for practical application and *Constraint logic programs*, or CLP, today are used in many different areas, particularly in Artificial Intelligence. It is worth noting that most of the current Prolog

³ Typically D is the set of real numbers.

implementations include an extension which allows for constraints, so most Prolog systems today are CLP languages.

A constraint can be seen as a particular first-order logic formula (normally we use a conjunction of atomic formulæ) which uses only predefined predicates. As previously mentioned, logic programming uses a particular kind of constraints, namely conjunctions of equations over the Herbrand Universe.

The idea of constraint logic programming is that of replacing the Herbrand Universe by another computational domain which usually is symbolic, but can also be arithmetic in some cases, and which is suitable for the application area of interest. Constraints, rather than equations between ground terms, define specific relations over the values in the new domain under consideration. Correspondingly, the basic computational mechanism will no longer be the solution of equations between terms (using unification) but will use a constraint-solving mechanism, that is an appropriate algorithm for determining solutions to the constraints to be solved.

For example, if we are interested in linear inequalities over real numbers, we could use the predicate (constraint) symbols $\geq$ and $\leq$ with the expected meaning. It is important to observe that this meaning, being the “expected” one, does not need to be defined by specific clauses in our program. Indeed, it is rather fixed a priori by a suitable logical theory that provides the correct axiomatisation and that we assume given. This also means that, when it comes to solving a constraint which uses the predicates $\geq$ and $\leq$, it is not sufficient to use the rules of our program, but we need to use also specific algorithms borrowed from mathematics which allow us to manipulate and solve inequalities according to the meaning provided by the chosen axiomatisation. This point will be further clarified in the next subsection, when defining the semantics of CLP.

The principal advantage of this approach should be clear: we can integrate into the logic paradigm (in a semantically clean fashion) very powerful computational mechanisms that were developed in other contexts for specific domains (such as linear programming, operation research, etc.)

13.2.1 Syntax and Semantics of CLP

To provide some more precise details on Constraint logic programming we now define its syntax and semantics.

Let us start by defining a syntax for constraints.

Definition 13.3 (*Constraint*) Assume given a signature (Σ, Π) augmented with constraint symbols, where the set of predicates Π contains a set Π_c of constraint symbols, which are different from the other predicate symbols. Assume also that Π_c contains $=$. The constraints over the signature with terms (Σ, Π) are defined as follows:

1. if $t_1, \dots, t_n$ are terms over the signature Σ and $c \in \Pi_c$ is a constraint symbol of arity n , then $c(t_1, \dots, t_n)$ is a constraint;
2. *true* and *false* are constraints;
3. if C and D are constraints then $(C \wedge D)$ (also written C, D) is a constraint.⁴

The syntax of CLP is a straightforward modification of that one of logic programming, provided in Definition 12.3, to take into account constraints.

Definition 13.4 (*Constraint logic program, CLP*) Let us assume given a signature (Σ, Π) augmented with constraint symbols, where $\Pi_c \subseteq \Pi$ is the set of constraint symbols. Let $H, A_1, \dots, A_n$ be atomic formulæ defined on the signature $(\Sigma, \Pi \setminus \Pi_c)$ and let C be a constraint on (Σ, Π_c) .

A CLP clause is a formula of the form:

$$H :- C, A_1, \dots, A_n.$$

If $n = 0$, the clause is said to be a *unit*, or a fact, and the symbol $:-$ is omitted (but not the final full stop). A constraint logic program is a set of CLP clauses. A query (or goal) is a sequence of constraints and atoms $C, A_1, \dots, A_n$.

As it appears from the previous definition, a CLP program is simply a logic program where each clause can contain a constraint (note that the constraint C in the clause can be omitted, syntactically represented by choosing for C the *true* constraint).

The idea of a CLP computation is that constraints, introduced by clauses, are accumulated during the computation and are then simplified and solved by suitable derivation steps which use an underlying constraint solver. This is analogous to the case of logic programming, provided that rather than explicit substitutions one uses equation on terms, which are then solved by the unification algorithm.

To formalise this intuitive notion of semantics, we deviate from the previous chapter, and we use the notion of *transition system* which has been introduced in Chap. 2 as a general tool for defining the operational semantics of programming languages.

We first assume given a consistent first-order constraint theory (CT) which defines the meaning of the constraint predicate symbols. Such a theory includes Clark equality theory (CET) defining the meaning of $=$ as syntactic equality over Herbrand Universe.⁵ Hence, we introduce a specialised transition systems for CLP, defined as follows.

⁴ Here, we have considered only conjunction, following the practice of modern CLP languages. In principle, one could consider also other logical connectives.

⁵ In some cases we will overload the symbol $=$, and we will use it also with a different meaning (for example, equality on numbers). These different uses will be explicitly mentioned.

Definition 13.5 (CLP transition system) A state is defined as a pair $\langle G; C \rangle$ where G is a goal and C is a constraint.⁶

A CLP transition system (in the following transition system for short) for the program P is a pair $(S; \mapsto)_P$ where S is a set of states and $\mapsto$ is a binary relation over states, also called *transition relation*, defined inductively by the following transition rules:

Unfold If $H :- C, A_1, \dots, A_n$ is a fresh variant of a clause in P and

$$CT \models \exists ((A = H) \wedge C)$$

then

$$\langle A \wedge G; D \rangle \mapsto \langle G \wedge C \wedge A_1 \wedge \dots \wedge A_n; (A = H) \wedge D \rangle.$$

Failure If there is no (fresh variant of) clause $H :- C, A_1, \dots, A_n$ in P such that

$$CT \models \exists ((A = H) \wedge C)$$

then

$$\langle A \wedge G; D \rangle \mapsto \langle A \wedge G; false \rangle.$$

Solve If $CT \models \forall ((C \wedge D_1) \leftrightarrow D_2)$ then

$$\langle C \wedge G; D_1 \rangle \mapsto \langle G; D_2 \rangle.$$

A few explanations are in order, here.

In this operational semantics a state consist of the goal which has still to be evaluated (G), together with the (conjunction of) all the constraints which have been computed so far (D). Hence in the rule “unfold”, analogously to the logic programming case, once we have selected an atom A in the current goal $A \wedge G$, a computation (or derivation, or reduction, or unfolding) step is possible if we find a suitable clause in the program whose head unifies with A , under the value for the variables defined by the constraint C in the clause. This is exactly what the formula $CT \models \exists ((A = H) \wedge C)$ says, where the theory CT has the function of specifying the meaning of the constraints.⁷ As in the case of logic programming, the atom A to be rewritten can be any atom in the current goal (we have used here a simplified notation, which avoids mentioning the rules obtained by considering the commutative nature of $\wedge$).

Rule “failure” describes what happens when we cannot find in the program such a clause: the computation fails, as it is for logic programs.

Rule “solve” instead introduces the major innovative feature of CLP, namely the constraint solving operation. As previously mentioned, different constraint domains, and therefore different theories CT , use different solvers with different internal

⁶ For better clarity, in states we use the symbol $\wedge$ to denote conjunction rather than the comma, which appear in the syntax of programs.

⁷ Note that the case of logic programming could be obtained from this general case by simply considering the only theory CET rather than a generic CT.

computations and a different syntactic manipulation of the constraints. However, all the different constraint system and solvers share the purpose of simplifying the constraints in order to reduce the search space and to obtain a solution. Certainly all these simplifications must be logically sound, that is, must reduce a constraint to another one, possibly simpler and closer to the solution, while preserving the logical meaning of the constraint itself. Hence, rule “solve” which states that, whenever we select in the current goal $C \wedge G$ a constraint C for evaluation, we can perform a derivation step which modifies (and simplifies) the constraint C and the constraint computed so far D_1 thus producing a new constraint D_2 , provided that this new constraint is still logically equivalent to $C \wedge D_1$, as expressed by the condition $CT \models \forall ((C \wedge D_1) \leftrightarrow D_2)$.

Once we have introduced the transition systems, the definition of computation (or derivation) is pretty standard⁸:

Definition 13.6 (CLP Computations) Assume given a transition system $(S; \mapsto)_P$ for the CLP program P . An *initial state* has the form $\langle G; true \rangle$, a *successful final state* has the form $\langle true; C \rangle$ with C different from *false*, while a *failed final state* has the form $\langle G; false \rangle$.

A *finite computation* (or derivation) is a sequence of states $S_1, S_2, \dots, S_n$ such that $S_i \mapsto S_{i+1}$ for $i \in [1, n - 1]$.

A *successful computation* has the form $S_1, S_2, \dots, S_n$ where S_1 is an initial state and S_n is a successful final state. In this case, if S_1 is the state $\langle G; true \rangle$ and S_n is the state $\langle true; C \rangle$ we say that C is the *computed constraint* for the goal G in the program P .

A *finitely failed computation* has the form $S_1, S_2, \dots, S_n$ where S_1 is an initial state while S_n is a failed final state.

Note that, as already mentioned, in CLP an answer is a constraint and, as a such, it can provide a set of possible values rather than a single value. Next subsection will provide a significative example. Moreover, also in the case of CLP we have two forms of non-determinism. These are completely analogous to those of logic programming and therefore their discussion is omitted.

13.3 Generate and Test Versus Constraint and Generate

So far we have seen CLP as a smooth extension of logic programming, indeed many implemented Prolog system today includes libraries and support for constraint programming. There is however a major difference between CLP and (traditional) Prolog,

⁸ Here, we consider only finite computations. Of course, infinite computations can be defined as well.

which affects the style of programming and, most important, the efficiency of the programs.

As we have seen in the previous chapter, arithmetics is rather clumsy in Prolog, in particular many arithmetic built-ins require that their arguments are ground in order to be evaluated. For example, while the evaluation of goal $p(Z)$ with the program

```
p(X):- Y =1, X is Y + 1.
```

produces that answer $Z=2$ as expected, the same goal in the program

```
p(X):- X is Y + 1, Y =1.
```

produces the error

```
Arguments are not sufficiently instantiated
```

because when the predicate `is` is evaluated the variable Y is not yet bound to 1.

This feature of arithmetic built-ins is the main reason for the “generate and test” style of programming that is common with standard Prolog: when we have to solve a problem where among the many different values for some variables we must select those which satisfy a given set of constraints, being the constraints expressed in terms of built-ins the only possible approach is first *generating* a selection of values for the variables and then *testing* whether those values satisfy the constraints. If not, new values are generated and tested and so on, until either we find a solution or we terminate the available values. This methodology uses some how a “brute force” approach which does not exploit the power of constraint programming and, in particular, the constraint propagation techniques which are behind several solvers (especially on finite domains).

Indeed, as mentioned before, in many cases the constraint themselves can be used to guide the selection of the right values, thus pruning in a considerable way the solution search space. The general idea of the “constraint and generate” style is then to *constraint* first the admissible values by using the problem constraints, and then *generate* the values chosen from the set of values which has been already reduced by the constraints. This approach is possible because, differently from Prolog, in CLP the computation proceed by accumulating constraints which may include non instantiated variables. The pruning of the search space obtained with the constraint and generate approach makes in general the computation much more efficient than with the generate and test approach and, in many cases, transforms a computationally unsolvable problem into a feasible one.

As a simple example illustrating these two different programming methodologies, consider the crypto-arithmetic problem consisting in solving the equation

```

  S E N D
+  M O R E
-----
= M O N E Y

```

where each letter represents a different digit.

This problem can be solved by a simple CLP program which exploits the constraint and generate methodology. The program, as well the other ones in the following, is derived from [2] and can be executed on SWI Prolog.

```
:- use_module(library(clpfd)).

send([S,E,N,D,M,O,R,Y]) :-
    gen_domains([S,E,N,D,M,O,R,Y],0..9),
    S #\= 0, M #\= 0,
    all_distinct([S,E,N,D,M,O,R,Y]),
    1000* S + 100* E + 10*N + D
    + 1000* M + 100* O + 10*R + E
    #= 10000* M + 1000* O + 100* N + 10*E + Y,
    labeling([], [S,E,N,D,M,O,R,Y]).

gen_domains([],_).
gen_domains([H|T],D) :- H in D, gen_domains(T,D).
```

Previous program uses the library `clpfd` which allows us to use a finite domain solver and several related functionalities. In particular, the predicate `in\2` checks that the first argument is an element of the domain appearing as second argument, where the range domain `0..9` denotes all the integer numbers greater or equal to 0 and smaller or equal to 9.

The program solves the problem by using the usual declarative approach. After having introduced the domain `0..9` for all the variables in the list `[S,E,N,D,M,O,R,Y]` (by using the predicate `gen_domains`), the constraints which model the problem at hand in term of equations and inequations are introduced: The constraints

$$S \neq 0, M \neq 0$$

require that the most significative digit cannot be zero, while the equation

$$\begin{aligned} &1000 * S + 100 * E + 10 * N + D \\ &+ 1000 * M + 100 * O + 10 * R + E \\ &\#= 10000 * M + 1000 * O + 100 * N + 10 * E + Y, \end{aligned}$$

provides the arithmetic formulation of the problem itself. The fact that different letters correspond to different digits is guaranteed by the global constraint

```
all_distinct([S,E,N,D,M,O,R,Y]).
```

However, using only these constraints the constraint solver cannot enforce a single solution since some variables have a range of several admissible values. In order to obtain a specific value for the variables then, we need to use the `labeling\2` built-in which essentially set the variables appearing in its second argument to all possible values compatible with the domain, backtracking in case the set values do not satisfy the constraints.⁹

The constraint and generate approach here emerges from the fact that the setting of values, done by the

```
labeling([], [S,E,N,D,M,O,R,Y])
```

⁹ The first argument of `labeling\2` specifies some options on the generation of the values.

call, is done *after* the introduction of the constraints in the computation.

It is useful to compare the execution time of previous program with that one of the following programs, which implements a generate and test approach, since the values for the variables (through the call to the `labeling` built-in) are now set *before* the introduction of the constraints.

```
:- use_module(library(clpfd)).

send([S,E,N,D,M,O,R,Y]) :-
    gen_domains([S,E,N,D,M,O,R,Y],0..9),
    labeling([], [S,E,N,D,M,O,R,Y]),
    S #\= 0, M #\= 0,
    all_distinct([S,E,N,D,M,O,R,Y]),
    1000*S + 100*E + 10*N + D
    + 1000*M + 100*O + 10*R + E
    #= 10000*M + 1000*O + 100*N + 10*E + Y.

gen_domains([],_).
gen_domains([H|T],D) :- H in D, gen_domains(T,D).
```

By using the `time` functionality of SWI Prolog one can check that while the execution of the goal

```
?- send([S,E,N,D,M,O,R,Y]).
```

in the first program produces the solution

```
S= 9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2
```

almost instantaneously (on the online SWI Prolog system), the same goal in the second program produces the answer

```
Time limit exceeded
```

and fails to find the solution. This provide a clear evidence of the computational advantage of the constraint and generate approach.

13.3.1 A Further Example

To get an idea of the possibilities of constraint logic programming, consider the problem of defining the amount of the instalments of a loan. The variables involved in this problem are the following: *F* is the requested financing, or the initial sum loaned, *NumR* is the number of the instalment, *Int* is the amount of interest, *Rate* is the amount of a single instalment and, finally, *Deb* is the remaining debt. The relation between these variables is intuitively the following. In the case in which there is no amount paid back (or that *NumR* = 0), clearly the debt is equal to the initial financing:

```
Deb = Fin.
```

In the case of the payment of a single instalment, we have the following relation:

$$\text{Deb} = \text{Fin} + \text{Fin} * \text{Int} - \text{Rate}$$

Here, the remaining debt is the initial sum to which we have added the interest accrued and have subtracted the amount repaid in an instalment. In the case of two instalments, matters are more complicated because interest is calculated on the remaining debt and not on the initial sum. The remaining debt, then, becomes the new financing. Using the NuFin1 and NuFin2 variables to indicate this new value of financing, we then have therefore the relation:

$$\begin{aligned}\text{NuFin1} &= \text{Fin} + \text{Fin} * \text{Int} - \text{Rate} \\ \text{NuFin2} &= \text{NuFin1} + \text{Fin} * \text{Int} - \text{Rate} \\ \text{Deb} &= \text{NuFin2}\end{aligned}$$

In the general case, we can reason recursively. We have therefore the following constraint program, which we will call `loan`:

```
:- use_module(library(clpr)).

loan(Fin, NumR, Int, Rate, Deb) :-
    {NumR = 0,
     Deb = Fin}.

loan(Fin, NumR, Int, Rate, Deb) :-
    {NumR >= 1,
     NuFin = Fin+Fin*Int-Rate,
     NuNumR = NumR-1},
    loan(NuFin, NuNumR, Int, Rate, Deb).
```

Understanding this program, given what we have already said, should not be too difficult. The first line tell us that here we are using a CLP system defined over the domain of real numbers, with the associated solver and functionalities.¹⁰

Previous program can be used, just like logic programs, in many ways. For example, if we take out a loan of 1000 Euros and having paid 10 instalments of 150 Euros each at 10% interest, we can use it to find out what the remaining debt is. The goal to solve this problem is:

```
?- loan(1000, 10, 10/100, 150, Deb).
```

When evaluated (with the previous program) this goal produces the result `Deb = 203.128`¹¹

We can also use the same program in an inverse fashion. That is, rather finding out what the requested funding was, knowing that we have paid the same 10 instalments at 150 Euros each, again at 10% interest, but with residual debt of? 0. The goal in this case is:

¹⁰ The loan program is written using the syntax required by the `clpr` library as used in SWI Prolog. The curly brackets in the rules are required by the syntax of `clpr` on SWI Prolog to separate constraints.

¹¹ Here and in the following the results are obtained on SWI Prolog online.

```
?- loan(Fin, 10, 10/100, 150, 0).
```

This yields the result $\text{Fin} = 921.685$.

Finally, we can also leave more than one variable in the goal, for example to determine the relation that holds between financing, instalment and debt, knowing that we pay 10 instalments at 10% interest. We can therefore formulate the query:

```
?- loan(Fin, 10, 10/100, Rate, Deb)
```

With appropriate assumptions on the constraint solver¹² we obtain the result:

```
Deb = 2.593 * Fin - 15.937 * Rate.
```

That is—and this is unique to the various paradigms that we have seen so far—it allows us to obtain a relation on the numerical domains as a result.

13.4 MiniZinc

In this section we briefly outline the main features of MiniZinc [3], a free and open-source constraint modeling language which can be used to model constraint satisfaction and optimization problems in a high-level and solver-independent way. Here we aim only at giving a taste of how constraint programming can be embedded in the context of an imperative setting, without any claim of completeness.

MiniZinc models¹³ are contained in files having the extension `.mzn` and are compiled to FlatZinc, a solver input language that is understood by many different solvers. Data can be contained in the MiniZinc models or in separate files having the extension `.dzn`. Some of the examples of MiniZinc models that we will use in the following are from the Handbook of MiniZinc [4].

13.4.1 A MiniZinc CSP Model

A MiniZinc model is composed of four main parts that we will now illustrate by using the same crypto-arithmetic example seen in Sect. 13.3.

First, the libraries that will be used in the model are introduced by using the `include` command. For example, the line

```
include "globals.mzn";
```

¹² Systems defined over real numbers, can require fairly substantial modifications even if the basic idea is always the same. In particular, the possibility of obtaining the results just discussed depends a great deal on the power of the constraint solver adopted by the CLP language.

¹³ Sometime the terminology “program” is also used instead of “model”. The latter however is more correct, since MiniZinc models lack many traditional parts of standard programs.

is used to include all the global constraints defined in the file `globals.mzn`. One of such constraints is `alldifferent`, which has exactly the same meaning seen before in Sect. 13.3.

Second, variables and parameters are specified with their types, as in

```
var 1..9: S;
var 0..9: E;
var 0..9: N;
var 0..9: D;
var 1..9: M;
var 0..9: O;
var 0..9: R;
var 0..9: Y;
```

where the integer range types `1..9` and `0..9` (with the obvious meaning) are used for the (global) variables `S`, `E`, ... This is similar to the declarations of variables that one find in the imperative languages, however there is an important difference: in MiniZinc we have *decision variables* (also called variables tout court) and *parameters*. Decision variables, which are declared by using the keyword `var` (as before), are normally assigned by the solver when searching for a solution or by the user with an explicit constraint of the form

```
constraint <variable> = <expression>;
```

Parameters are declared by either using the keyword `par` or omitting any specific keyword. A parameter can be assigned explicitly a fixed value in the model or in the data file, very much as for a variable in imperative programs with the notable difference that a parameter cannot be assigned twice.

Third we introduce in the model the constraints, which have the form

```
constraint <Boolean expression>;
```

where in Boolean expression we can use the usual arithmetic operators (+, -, * ...), comparison predicates (>, <, ...), logical connectives ($\neg$, $\wedge$, $\vee$, $\rightarrow$, ...) global constraints and other constraints. In our example we can use the constraints

```
constraint 1000 * S + 100 * E + 10 * N + D
          + 1000 * M + 100 * O + 10 * R + E
          = 10000 * M + 1000 * O + 100 * N + 10 * E + Y;
```

```
constraint alldifferent([S,E,N,D,M,O,R,Y]);
```

with the obvious meaning.

Finally we must include in our model one (and only one) “solve” item which allow us to specify whether we are solving a satisfiability problem, and therefore we are just looking for one solution, or we are solving and optimization problem, and therefore we are looking for the optimal solution. Solve item has then one of the following three forms:

```
solve satisfy;
solve maximize <arithmetic expression>;
solve minimize <arithmetic expression>;
```

In our example we simply need one solution, so we write

```
solve satisfy;
```

To present the results of the model execution in a more nice way we can use output items of the form:

```
output [<string expression>, ... , <string expression>];
```

where the string expressions provide some specific instructions for presenting the results.

Putting together all the components of our example, and adding the output part, we obtain the model

```
include "globals.mzn";

var 1..9: S;
var 0..9: E;
var 0..9: N;
var 0..9: D;
var 1..9: M;
var 0..9: O;
var 0..9: R;
var 0..9: Y;

constraint 1000 * S + 100 * E + 10 * N + D
          + 1000 * M + 100 * O + 10 * R + E
          = 10000 * M + 1000 * O + 100 * N + 10 * E + Y;

constraint alldifferent([S,E,N,D,M,O,R,Y]);

solve satisfy;

output ["␣(S)␣(E)␣(N)␣(D)␣n",
        "␣␣(M)␣(O)␣(R)␣(E)␣n",
        "␣␣␣(M)␣(O)␣(N)␣(E)␣(Y)␣n"];
```

which, once have been saved on a file, call it send.mzn, can be solved on any system supporting MiniZinc to obtain the following result:

```
Compiling send.mzn
Running send.mzn
  9567
+ 1085
= 10652
-----
Finished in 174msec
```

It is worth noticing that, differently from traditional imperative languages, the ordering of the various parts of the model is not relevant: for example, constraints could be defined before parameters and variables.

13.4.2 A MiniZinc COP Model

We now show an example of an optimization model by using a simplified version of the Traveling Salesperson Problem (TSP). In doing so, we will introduce some more features of MiniZinc (these features, of course, can be used also in satisfaction problems).

Concerning data, we can use *arrays*, both of variables and of parameters, in a fashion very much similar to other imperative languages (perhaps with the exception that here array indexes start from 1).

```
int: n;                % number of cities
int: start_city;       % the city where we start the tour
int: end_city;         % the city where we end the tour

array[1..n] of string: city_name; % names corresponding to
                                % integers denoting cities
array[1..n,1..n] of int: distance; % city distances
array[1..n] of var 1..n: solution; % order of cities in solution
```

Lines starting with % in the code above are comments which should help understanding the model. Here we introduce three parameters `n`, `start_city` and `end_city`, two arrays of parameters `city_name` and `distance` and one array of variables `solution`. The problem consists in finding a path for visiting all the `n` given cities once, from `start_city` to `end_city`, while minimizing the total distance travelled, where the distance between two next cities in the path is provided in the matrix `distance`.¹⁴ The (array of) parameters will be instantiated by the data provide as input to the problem, while the array of variables will be instantiated by the solver and will provide the solution in the form of a sequences of cities which will form our travel.

Next we introduce the constraints, which simply state which are the start and end city of the travel in the solution, and that no city can be visited more than once. Notice that this last constraint is enforced by simply using as argument to `all_different` the name of the array whose elements must be different (of course, we must include the appropriate library to be able to use this global constraint).

```
constraint solution[1] = start_city;
constraint solution[n] = end_city;
constraint all_different(solution); % no city visited more than once
```

¹⁴ Here we assume that every city is connected to all other cities by a direct path (an arc in the graph). This assumption can be relaxed easily, by introducing a max value which is greater than the sum of all other distances to represent the absence of a direct path between two cities, and then checking accordingly the found solution. Moreover, normally the TSP problem requires that the end city is the same as the start city while here we relax this constraint.

Finally we must define the objective function which need to be minimized and then call the solver:

```
var int: total_distance = sum(i in 2..n)
                        (distance[solution[i-1],solution[i]]);
solve minimize total_distance;
```

Here we use the MiniZinc aggregation function `sum` which allows to sum all the elements of an array for a given range of the index. In our case, we are summing the values of the matrix `distance` in position `[solution[i-1],solution[i]]`, for all the values of the index `i` contained in the range `2..n`. This means we are summing the distances from the first city of the tour to the second, from the second to the third and so on until the end city. It is also worth noticing that the indices `solution[i-1]` and `solution[i]` of the distance matrix are variables which will be instantiated by the solver when this has found a solution.

MiniZinc also supports the following aggregation functions for arithmetic arrays; `product`, which multiplies the elements of the array, `min` and `max` which respectively return the least and greatest element in the array. For arrays containing Boolean expressions MiniZinc offers four other aggregation functions: `forall` returns the logical conjunction of all the constraints in the array, thus guaranteeing that all the constraints hold, while `exists` returns the logical disjunction of all the constraints in the array, thus guaranteeing that at least one constraint hold. Moreover we have also `xorall` and `iffall` which ensure that an odd or even number of constraints hold, respectively.

We can now put together all the components of our TSP model, and by adding some cosmetic aspects for the output we obtain:

```
include "all_different.mzn";

int: n;                % number of cities
int: start_city;        % the city where we start the tour
int: end_city;          % the city where we end the tour

array[1..n] of string: city_name;% names corresponding to
                        % integers denoting cities
array[1..n,1..n] of int: distance; % city distances
array[1..n] of var 1..n: solution; % order of cities in solution

constraint solution[1] = start_city;
constraint solution[n] = end_city;
constraint all_different(solution);% no city visited more than once

var int: total_distance = sum(i in 2..n)
                        (distance[solution[i-1],solution[i]]);
solve minimize total_distance;

output[show(solution)];
output [city_name[fix(solution[i])] ++ "_->_" | i in 1..n ] ++
      ["\nTotal hours travelled: ", show(total_distance) ];
```

If we save this model in a file `tsp.mzn`, add to it the following data (either in the same `tsp.mzn` file or in a separate `.dzn` file)

```
n=5;
distance=[|0,2,3,6,8|2,0,1,4,6|3,1,0,3,5|6,4,3,0,2|8,6,5,2,0|];
start_city=2;
end_city=1;
city_name=["milano","bologna","firenze","roma","napoli"];
```

and we then we run `tsp.mzn` we obtain the result:

```
Compiling tsp.mzn
Running tsp.mzn
[2, 5, 4, 3, 1]bologna -> napoli -> roma -> firenze -> milano ->
Total hours travelled: 14
-----
=====
Finished in 73msec
```

13.5 Summary

In this chapter we have introduced the notion of constraint and we have then discussed the main programming paradigms which allow us to use constraints in a native way. In particular we have focused on Constraint logic programs (CLP), a smooth integration of Constraint programming and logic programming which retains the advantages of both the approaches. We have also discussed the use of constraint programming in the context of an imperative language by considering MiniZinc, a free and open-source constraint modeling language which can be used to model both constraint satisfaction and optimization problems in a high-level and solver-independent way.

The use of sophisticated constraint solvers and also optimisers (for example, based on the Simplex algorithm) makes constraint programming competitive with respect to other formalisms in many commercial applications where solutions to combinatorial problems, solutions to optimisation problems and more generally solutions to problems expressible as relations over appropriate domains are required. Examples in this sense include a variety of problems typical of artificial intelligence, from traditional scheduling problems and bioinformatics, to modern decision support systems and fintech applications. The bibliographical notes in the next section provide references to many more examples.

13.6 Bibliographical Notes

Solving an equation over the integers is a form of constraint solving and is one of the oldest problems in mathematics—already the Babylonians were able to solve several equations with two variables. So it is not surprising that there exists a huge literature on constraint solving and programming. Here we mention only a few publications

which are directly relevant to the content of this chapter and which can also be consulted for finding references to more specialised material.

Constraint satisfaction problems arose in Artificial Intelligence already in the 70s of previous century, in particular for applications to picture processing, see for example the article [5] by Montanari which introduced also the notions of node and arc consistency.

The first true constraint logic programming language was PROLOG II, a language designed by Colmerauer in 1982 [6] which allowed equations and disequations over rational trees. Later on in 1986 Colmerauer extended PROLOG II to PROLOG III [7], a language which provided also constraints on booleans, strings and arithmetics. Jaffar, Lassez and Maher showed in [8] how the usual equation over terms of logic programming could be extended to constraints over generic domains while maintaining the traditional semantics of logic programming, a result later obtained also for other more operational semantics [9].

The paper [10] in 1987 clarified that these and other extensions of logic programming were instances of the constraint logic programming scheme, thus introducing the terminology “Constraint Logic Programming”. Jaffar and his colleagues at Monash university in these years developed also CLP($\mathcal{R}$) [11, 12], a CLP language over real arithmetic, while Dincabas and others at ECRC in the same period developed CHIP [13], an extension of Prolog which allowed constraints over finite domains and integer ranges. The survey [14] provides an excellent overview of theory and applications of CLP, including an historical perspective and a large number of references.

The books [2] by Marriott and Stuckey, [15] by Frühwirth and Abdennadher, and [16] by Apt all provide very good introductions to languages with constraints, in particular [2] focus on CLP.

The documentation available at [3] can be consulted for a more in depth introduction to MiniZinc while the recent book [17] by Wallace introduces the principles of Decision Support Systems and illustrate how to build them by using MiniZinc.

13.7 Exercises

1. Consider the following program

```
:- use_module(library(clpr)).
p(X,Y):- {X=0, Y=0}.
p(X,Y):- {Y = X*X+Z}, p(X-1, Z).
```

Show a derivation for the goal

```
?- p(2,W).
```

and provide the answer computed by such a derivation.

2. Consider the following program

```
:- use_module(library(clpr)).
```

```
p(X,Y):- {X>=1, Z>=0, Y = X+Z}, p(X-1, Z).
p(X,Y):- {X=0, Y=0}.
```

What is the answer for the following goal?

```
?- p(W, 3).
```

Consider now the program

```
:- use_module(library(clpr)).
p(X,Y):- {X>=1, Y = X+Z}, p(X-1, Z).
p(X,Y):- {X=0, Y=0}.
```

And the same goal as before

```
?- p(W, 3).
```

What happens now in the derivation for this goal?

3. Antonio, Mary, Carlos, and David are eating a cake and must divide the 9 slices between each other. Antonio cooked the cake, so he wants more slices than anyone else. Mary has done her workout in the morning, so she deserves a treat and wants at least 3 slices. Carlos is on a diet, so he will eat less than 3 slices. David wants to feel unique, so he will eat a number of slices that is different from anyone else, and at least 1. They want to save the remaining slices in the fridge, but the fridge is almost full, so only 1 slice can remain.

Write a CLP or a MiniZinc program to compute how they can divide the slices.

4. Morgan is a traveling salesperson that must visit three clients: Luca, Beth, and Mario. The house of Morgan is distant 3 h from each client. The house of Luca is distant 2 h from Beth, and 3 from Mario. The houses of Beth and Mario are only 1 h distant from each other. Beth must be visited before 14, because she is very busy in the afternoon.

Knowing that Morgan won't leave the house before 9 and want to be back at home at most at 18, write a CLP or a MiniZinc program to compute the possible timetables for Morgan, including leaving and returning home.

5. A traveller is planning her next hike. Starting from her current base, she plans to reach the top of a mountain and then descend to a village. There are two possible path to reach the mountain, the path A requires 2 h, the path B requires 3 h. Then it is possible to take the path C to directly reach the village in 4 h. Alternatively, it is possible to visit another mountain, following the path D, in 1 h. From the second mountain, it is possible to reach the village in 3 h following the path E, or 4 h following the path F. The traveller wants to complete the hike in strictly less than 8 h. Write a CLP or a MiniZinc program to compute which choices traveller has and, for each possible path, what is the required time.
6. A scheduling problem consists in a defining an appropriate scheduling for executing a number of tasks, while observing the given durations of each task and the precedence constraints among them (e.g. painting must be performed after the wall has been constructed).

Write a CLP or a MiniZinc program to solve a scheduling problem, where we have the following tasks and durations:

Task	Duration
t1	12
t2	6
t3	11
t4	7
t5	5
t6	14

and we have the following precedence relation

t1 before t4
t2 before t5
t3 before t4
t4 before t5

Hint: represent each task by a different variable which specifies the starting time unit of the task. Precedence constraints among the tasks can be encoded as inequalities among the corresponding task variables.

7. (Zebra puzzle). A small street has five differently colored houses on it. Five persons of different nationalities live in these five houses. Each person has a different profession, each person likes a different drink, and each has a different pet animal. Moreover we know the following:

- The English lives in the red house.
- The Spaniard has a dog.
- The Japanese is a painter.
- The Italian drinks tea.
- The Norwegian lives in the first house on the left.
- The owner of the green house drinks coffee.
- The green house is on the right of the white house.
- The sculptor breeds snails.
- The diplomat lives in the yellow house.
- They drink milk in the middle house.
- The Norwegian lives next door to the blue house.
- The violinist drinks fruit juice.
- The fox is in the house next to the house of the doctor.
- The horse is in the house next to the house of the diplomat.

We want to know who has the zebra and who drinks water.

Write a CLP or a MiniZinc program to solve the Zebra puzzle by using finite domains.

References

1. E.C. Freuder, A constraints journal. *Constraints* **1**(1), 5 (1996)
2. K. Marriott, P.J. Stuckey, *Programming with Constraints* (MIT Press, 1998)
3. Monash University and Data61 CSIRO. MiniZinc home page (2014). <https://www.minizinc.org/>. Accessed 05-January-2023
4. P.J. Stuckey, K. Marriott, G. Tack, *The MiniZinc Handbook* (2015). www.minizinc.org/doc-2.6.4/en/index.html. Accessed 05-January-2023
5. U. Montanari, Networks of constraints: fundamental properties and applications to picture processing. *Inf. Sci.* **7**, 95–132 (1974)
6. A. Colmerauer, Prolog II reference manual and theoretical model. Technical report (Université Aix-Marseille II, Groupe Intelligence Artificielle, 1982)
7. A. Colmerauer, Note sur Prolog III, in *SPLT'86, Séminaire Programmation en Logique, 21-23 mai 1986, Trégastel, France* (1986), pp. 159–174
8. J. Jaffar, J. Lassez, M.J. Maher, Logic programming language scheme, in *Logic Programming: Functions, Relations, and Equations*, ed. by D. DeGroot, G. Lindstrom (Prentice-Hall, 1986), pp. 441–467
9. M. Gabbrielli, G. Levi, Modeling answer constraints in constraint logic programs, in *Logic Programming, Proceedings of the Eight International Conference, Paris, France, June 24-28, 1991*, ed. by K. Furukawa (MIT Press, 1991), pp. 238–252
10. J. Jaffar, J. Lassez, Constraint logic programming, in *Conference Record of the Fourteenth Annual ACM Symposium on Principles of Programming Languages, Munich, Germany, January 21–23, 1987* (ACM Press, 1987), pp. 111–119
11. J. Jaffar, S. Michaylov, Methodology and implementation of a CLP system, in *Logic Programming, Proceedings of the Fourth International Conference, Melbourne, Victoria, Australia, May 25–29, 1987 (2 Volumes)*, ed. by J.-L. Lassez (MIT Press, 1987), pp. 196–218
12. J. Jaffar, S. Michaylov, P.J. Stuckey, R.H.C. Yap, The CLP(R) language and system. *ACM Trans. Program. Lang. Syst.* **14**(3), 339–395 (1992)
13. A. Aggoun, M. Dincbas, A. Herold, H. Simonis, P. Van Hentenryck, The CHIP system. Technical Report TR-LP-24 (European Computer Industry Research Centre (ECRC), Munich, Germany, 1987)
14. J. Jaffar, M.J. Maher, Constraint logic programming: a survey. *J. Log. Program.* **19**(20), 503–581 (1994)
15. T.W. Frühwirth, S. Abdennadher, *Essentials of Constraint Programming, Cognitive Technologies* (Springer, 2003)
16. K.R. Apt, *Principles of Constraint Programming* (Cambridge University Press, 2003)
17. M. Wallace, *Building Decision Support Systems - Using MiniZinc* (Springer, 2020)